

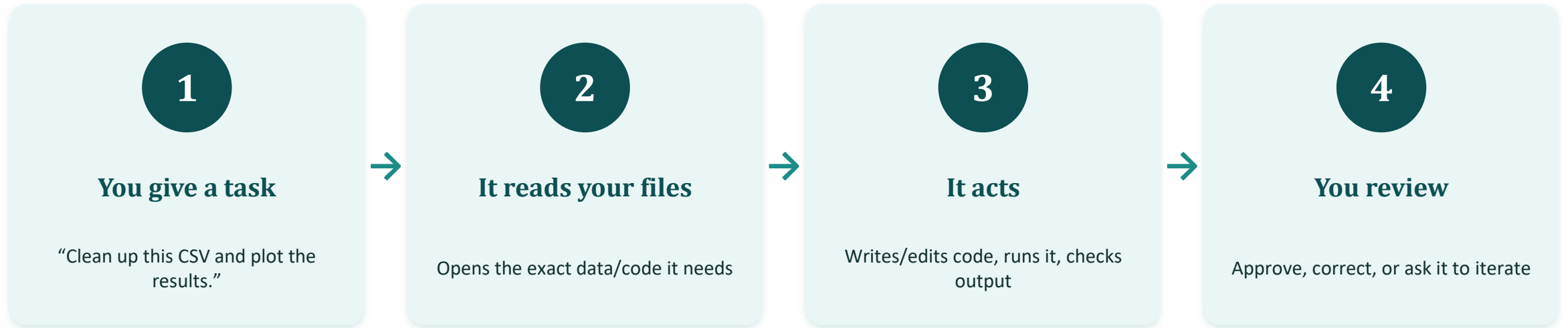
Claude Code

A 20-Minute Crash Intro for Research Trainees & Faculty

How it works • What it's for • Using tokens wisely

What Is Claude Code?

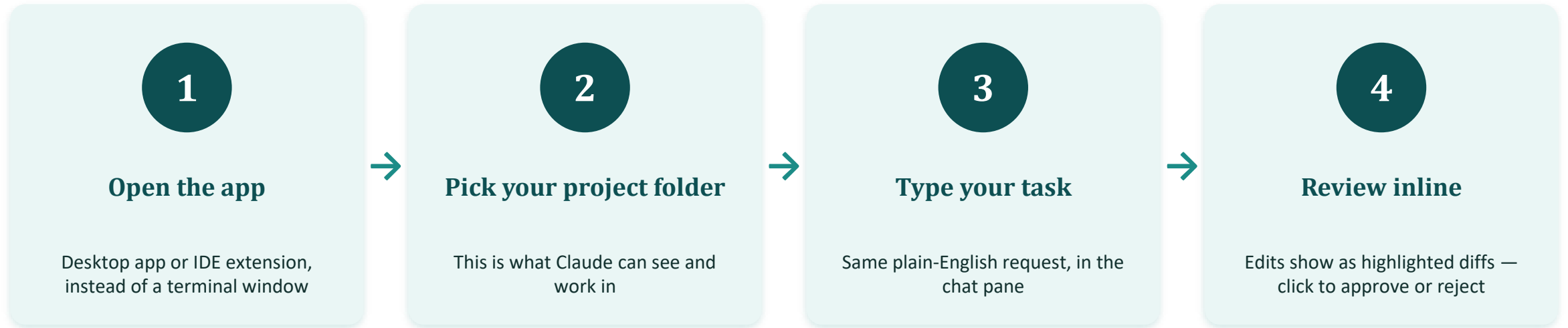
An agentic assistant that lives in your terminal — it reads files, writes and runs code, and iterates. Not just a chatbot.



Loop back to step 1 as many times as needed — that's the workflow.

No Terminal? Use the GUI App

Claude Code also runs as a desktop app / IDE extension — same tool, no terminal required.



Same underlying tool as the CLI — everything else in this deck applies either way.

Two Ideas Worth Understanding First



Context Window

What it “remembers” during a session: your conversation, plus any files it has opened.

Very long sessions get automatically summarized to keep going — but a focused, shorter session is faster and cheaper.

New task? Start a new session so old context doesn't linger.



You Stay in Control

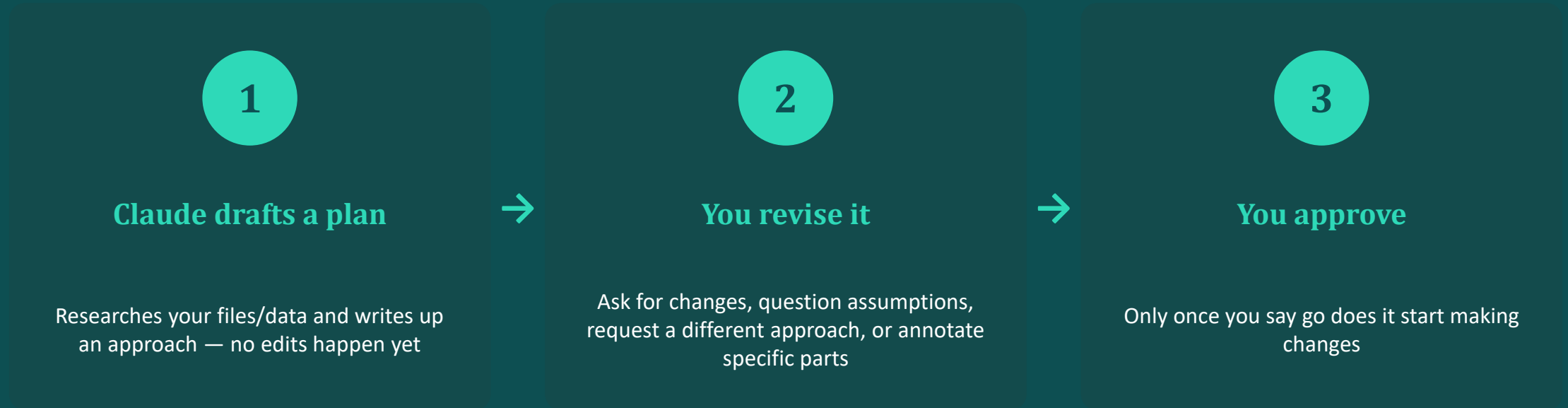
By default, Claude Code asks permission before risky actions — editing files, running commands, deleting anything.

You see exactly what it wants to do before it happens, and can approve, edit, or reject each step. In the GUI app, this shows up as a click-to-approve diff instead of a terminal prompt.

Nothing runs on your data or files without your say-so.

Plan Mode: Think Before You Touch

For big or ambiguous tasks, Claude drafts a plan first — nothing changes until you approve it.



Great for scoping a grant draft, an analysis approach, or any change worth getting right before spending tokens executing it.

Example: A Flow Cytometry Data Task

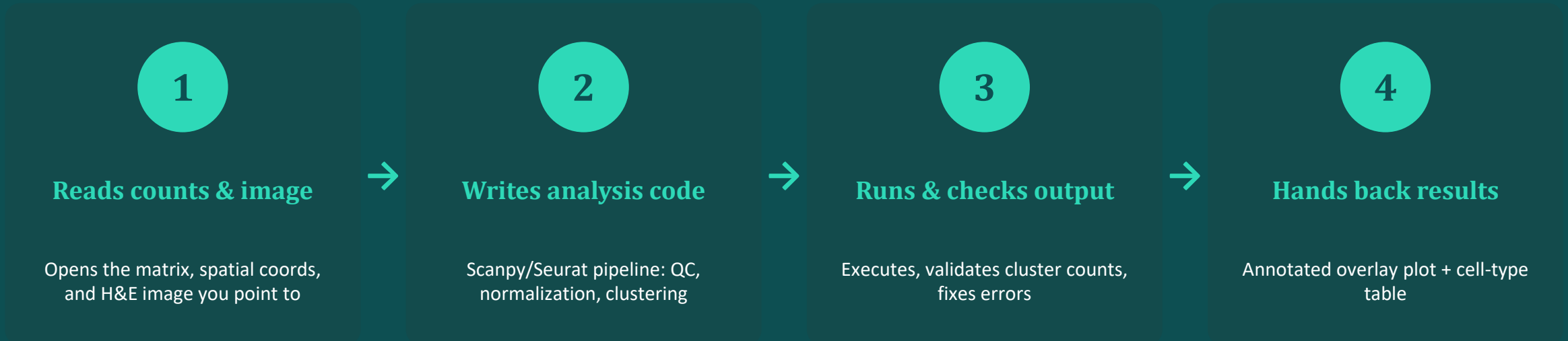
“Here's my flow cytometry export — gate on CD3+/CD8+ and give me a summary table.”



The same pattern works for plate-reader data, imaging pipelines, sequencing file cleanup, and more.

Example: Spatial Multiomics & Sequencing

"I have Visium spatial transcriptomics counts plus the matched H&E image — cluster spots, annotate cell types, and overlay results on the tissue."



Same approach for bulk/single-cell RNA-seq, ATAC-seq, and other sequencing pipelines — point to the files, let it write and run the code.

Where It Helps in the Lab



Analysis scripts

Process plate-reader data, stats, batch jobs



Data cleaning

Merge/standardize messy experiment sheets



Plots & QC

Dose-response curves, outlier flags



Gating & imaging

Flow cytometry, image batch processing



Sequencing & spatial

scRNA-seq, ATAC-seq, spatial multiomics



Pipeline glue

Rename/organize experiment files at scale



Explaining output

Plain-language read of stats/model output

Beyond the Bench

It helps with reading and writing research too, not just running it.



Paper review & critique

“Critique this manuscript's methods — are the statistical tests appropriate for this design?”



Explaining complex ideas & formulae

“Walk me through equation 3 in this paper in plain language, and show the derivation step by step.”



Grants & fellowships

“Help me tighten the specific-aims page against this funder's page limit without losing the key logic.”

What It's Not a Substitute For

It accelerates implementation — it does not replace domain judgment.

-  Scientific conclusions — you still decide what the data means
-  Statistical consulting — verify method choice with a statistician for high-stakes analyses
-  Domain expertise — it doesn't know your assay's quirks unless you tell it

Token Usage 101

Everything Claude reads or writes is measured in tokens — roughly 3/4 of a word each.

More tokens

= slower responses

More tokens

= higher cost per task

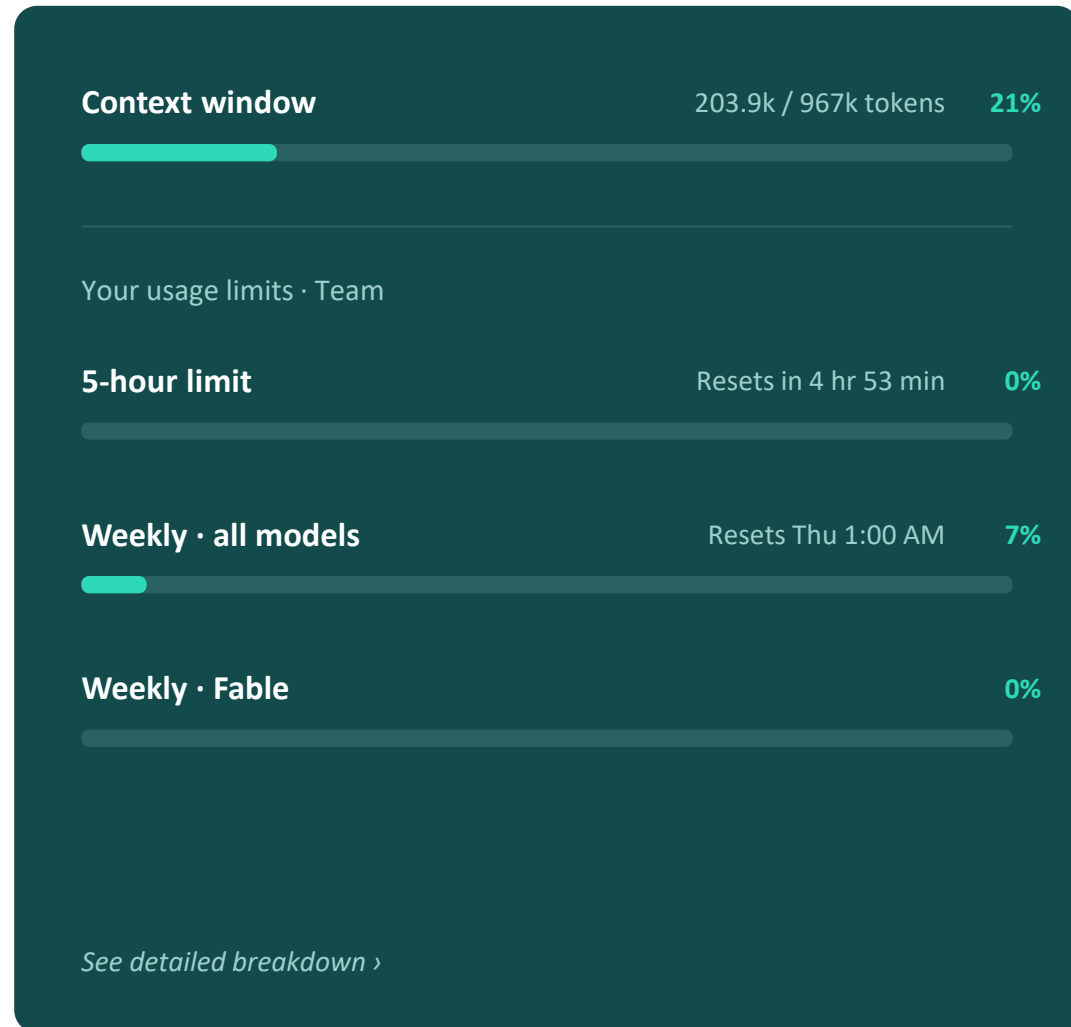
Fewer, focused tokens

= faster, cheaper, sharper answers

The good news: a few simple habits (next slides) cut token use a lot without losing quality.

Checking Your Usage

Claude Code: run `/usage`. • *claude.ai*: Settings → Usage. • Shown here as a stylized recreation of that panel.



Context window

How much of the current conversation + files fit in this one session — not a time-based limit. Runs out mid-conversation, not mid-week.



5-hour limit

A rolling budget shared across Claude.ai, Claude Code, and the desktop app. Resets 5 hours after your first message in the window.



Weekly limit

A separate fixed cap that resets on your assigned day. Applies even if your 5-hour budget still has room left.



Per-model weekly quotas

The highest-capability model (Fable) gets its own additional weekly quota on top of the shared “all models” quota, since it costs the most to run.

Model Tiers & Effort

Higher capability and higher effort both draw down your usage limits faster — match the tool to the task.

Model	Capability	Speed	Context window	Typical use
Fable	Highest	Slower	1M tokens	Long-running agents, deep multi-step research
Opus	Very high	Moderate	1M tokens	Complex agentic coding, large refactors
Sonnet	High (default)	Fast	1M tokens	Everyday coding, analysis, writing — best default
Haiku	Good, near-frontier	Fastest	200k tokens	High-volume, quick tasks, sub-agent helpers

Effort	What it controls
Low	Fastest, most token-efficient — simple/quick tasks, high-volume work
Medium	Balanced cost vs. quality — solid default for routine work
High	Full reasoning depth — default for most models on complex tasks
X-High / Max	Maximum reasoning for long, hard problems — highest token cost

Best Practice: Be Specific & Scoped



Costly & vague

“Look through my whole project and fix anything that seems off.”

Forces a broad search across every file — burns tokens before real work even starts.



Specific & scoped

“In analysis/clean_data.py, standardize the column names in results.csv.”

Claude reads exactly what's relevant and gets to the fix immediately.

Best Practice: Manage What It Reads



Point to files, don't paste data

Give the file path instead of pasting a huge CSV or log into chat
— it reads only what it needs.



New session per unrelated task

Old context you don't need still costs tokens to carry around.
Start fresh when the topic changes.

Best Practice: Plan Big, Check In Often



Use Plan Mode first

For large changes, let it draft a plan before writing code — cheaper to redirect a plan than redo a big wrong change.



Checkpoint on long tasks

Review progress periodically instead of letting it run unsupervised for a long stretch.



Prefer scripts over chat

A reusable script is cheap to keep; one-off manual work in chat isn't. Ask it to write the script.

Try It Yourself

Same tool either way — pick whichever fits how you already work.

Terminal (CLI)

1

Open a terminal

Navigate into your project folder

2

Type: claude

Starts a Claude Code session

3

Describe your task

Plain English — name the file if you know it

4

Review & iterate

Approve, correct, or ask it to keep going

GUI App

1

Open the app

Desktop app or IDE extension — no terminal needed

2

Pick your project folder

This is what Claude can see and work in

3

Type your task

Same plain-English request, in the chat pane

4

Review inline

Diffs are highlighted — click to approve or reject

Resources



Cheat sheet

One-page reference — keep it at your desk



Try a real task today

Pick one script or cleanup job you already need



Questions later

Reach out any time — happy to pair on a first task

Thank you — let's start building.